Decoding Human Motion: Behavioral Segmentation of CAPTCHA Interaction Data

Team 165

Seoyoung Cho
scho412@gatech.edu

Luke Hiura
lhiura3@gatech.edu

Younghyun Kweon
ykweon31@gatech.edu

Michael Yu
myu370@gatech.edu

Patrick Zailaa
pzailaa3@gatech.edu
Georgia Institute of Technology
Atlanta, Georgia, USA

Abstract

Behavioral CAPTCHAs increasingly exploit cursor dynamics for verification, yet few public corpora expose large-scale, session-level pointer traces under controlled task variation, which limits reproducible benchmarking and model comparison. In this work, we study CaptchaSolve30k [6], an open release of approximately 30,000 verified human sessions. We summarize each session with six kinematic descriptors, normalize features strictly within game type so that distances reflect movement strategy rather than raw puzzle difficulty, and apply PCA to the resulting z-scored vectors. A median consensus over standard clustering diagnostics selects K=4, and we fit k-means in the retained principal subspace to obtain interpretable movement archetypes. We then train a Random Forest that reproduces the unsupervised partition at inference time, export the full preprocessing and modeling bundle for replication, and connect a Flask service to a D3.js dashboard so analysts can inspect live sessions in the same PC projection as the offline cohort together with cohort-relative anomaly scores.

1 Introduction

Modern web security relies on CAPTCHAs to distinguish humans from automated bots. However, as generative AI and computer vision models now solve static image puzzles with near-human accuracy [5], the security frontier has shifted from *what* a user selects to *how* a user moves. Verification now increasingly depends on continuous cursor telemetry—capturing curvature, acceleration, and hesitation—rather than just the final click [15, 16].

Despite the critical importance of these "micro-movements," research in this area is stifled because large-scale behavioral logs are typically proprietary. This lack of transparency prevents independent replication and makes it difficult to establish a "ground truth" for human behavior. Furthermore, existing models often fail to account for task context: a slow, wandering cursor might signal a bot on a simple task, but it is perfectly normal for a human struggling with a complex one.

In this work, we address these gaps by using the publicly released CaptchaSolve30k benchmark [6] to characterize human session variability. Our motivation is to define stable behavioral archetypes that remain valid across different types of puzzles. By establishing these human baselines through within-task normalization and unsupervised clustering, we provide a foundation for more resilient security screenings. We conclude by operationalizing our findings into a real-time inference pipeline and an interactive visualization dashboard for qualitative behavioral analysis.

2 Problem Definition

2.1 Objectives (Jargon-Free)

Our goal is to identify and categorize the different "styles" of how people move their computer mouse when solving security puzzles (CAPTCHAs). Just as some people drive cars very aggressively while others are more cautious, computer users exhibit unique movement patterns—some move directly and quickly, while others are slower or take more curved paths.

We want to group these similar behaviors together so we can define what "normal" human movement looks like across different types of puzzles. Importantly, we aim to distinguish between a user who is moving slowly because they are hesitant and a user who is moving slowly simply because the puzzle they are solving is difficult. By creating these categories, we provide a baseline that helps website owners better understand user behavior and identify suspicious activity.

2.2 Formal Problem Statement

Let $\mathcal{S} = \{s_1, \ldots, s_N\}$ index sessions from CaptchaSolve30k. Each s_i is a time-ordered sequence of cursor samples (x_j, y_j, b_j) indexed by sample time or by `sampleIndex`, summarized by a fixed-length kinematic vector $\mathbf{f}(s_i) \in \mathbb{R}^6$ (Table 1).

Unsupervised learning partitions $\mathcal{S}$ into K clusters $\{C_1, \ldots, C_K\}$ intended to summarize distinct movement strategies; we then fit a Random Forest h such that $h(\mathbf{f}(s_i))$ reproduces the cluster label assigned to s_i by the chosen clustering. This agreement is measured with respect to the unsupervised partition itself rather than an external behavioral taxonomy. Throughout the deployed system we fix K=4 as motivated in §4.

The central modeling difficulty is to separate interpretable movement profiles from high-frequency noise while respecting that raw kinematic magnitudes are task-conditioned. A slow, circuitous trace can be typical under a difficult puzzle and anomalous under a trivial one, so features must be normalized within game type before distances are interpreted.

3 Literature Survey

3.1 CAPTCHA Evolution and Vulnerabilities

Behavioral CAPTCHAs build on von Ahn et al.'s notion of hard AI problems for security [15] and increasingly exploit continuous cursor metrics [16]; automated attacks on classical puzzles [5] reinforce interest in movement-derived signals. Much of the published evidence nevertheless rests on black-box or proprietary logs, which limits reproducibility and open comparison across systems.

3.2 Mouse Dynamics and Behavioral Biometrics

Mouse dynamics support behavioral biometrics: trajectory-derived features can discriminate users [3] and support continuous monitoring [13], while statistical regularities in pointing behavior enable verification [17]. Stragapede et al. [14] highlight the value of public behavioral corpora. Prior work on mouse dynamics largely focuses on authenticating known users rather than segmenting anonymous sessions into interpretable behavioral groups. Time-series clustering and elastic alignment of trajectories (e.g., dynamic time warping [4]; surveys of temporal clustering [2]) provide closer analogs to summarizing cursor streams than one-off pointing studies, and motivate our use of fixed-length kinematic summaries before partitioning.

3.3 Visual Analytics and Dimensionality Reduction

To effectively visualize these high-dimensional behavioral clusters, we draw upon Munzner's framework for visualization analysis [11] and Keim's visual analytics process [9]. Dimensionality reduction is critical for this; while techniques like UMAP [10] are popular for non-linear data, our early experiments indicated that linear combinations (PCA) are highly effective for our specific kinematic feature set, aligning with best practices for interactive graphics [12]. Off-the-shelf reduction does not by itself encode task semantics: for highly specific, task-dependent kinematics we therefore combine dimensionality reduction with strict within-task normalization so that distances in the reduced space reflect movement strategy rather than raw puzzle difficulty.

4 Proposed Method

4.1 Intuition

We prioritize interpretable movement summaries and numerically stable inference over maximizing opaque scores on surrogate labels. Normalizing each feature within game type removes systematic shifts induced by puzzle difficulty, so that clusters in the reduced space track variation in movement strategy rather than task identity alone. We select the number of clusters with a median consensus over complementary diagnostics on the PCA embedding, which reduces sensitivity to any single heuristic. PCA furnishes a low-dimensional Euclidean workspace in which k-means remains well behaved while retaining most variance in the z-scored kinematics. Finally, we distill the resulting partition with a Random Forest so that the same six-dimensional representation can be evaluated in milliseconds at inference time and projected consistently into the PC1–PC2 view that drives the dashboard.

4.2 Data Loading and Trajectory Cleaning

We use all 30,000 raw sessions from CaptchaSolve30k [6], loading the published Hugging Face splits and concatenating rows. Each session is a time-ordered stream of (x, y), button state, and *sampleIndex*. We deduplicate ticks by *sampleIndex* and drop steps whose displacement exceeds 800 px per sample to remove teleportation artifacts. From the remaining trajectories, we distilled the telemetry down to six specific kinematic features (Table 1). We deliberately selected this concise set over a richer, higher-dimensional feature space to preserve behavioral interpretability, keep extraction numerically stable, and meet low-latency constraints on the live Flask backend.

Table 1: Behavior Measures

Feature	What it measures
duration	Total session time in milliseconds
path_length	Total cursor distance traveled (px)
speed_mean	Average step distance (px/sample)
path_efficiency	Straight-line distance ÷ path length (1.0 = perfectly direct)
pause_rate	Fraction of steps where speed < 0.5 px/sample (near-stationary)
speed_std	Variability of speed — captures bursts vs. smooth movement

4.3 Data preprocessing: mouse-only cohort

Mixing pointer hardware can dominate low-level kinematics before any behavioral interpretation is attempted. An exploratory k=3 fit on *all* devices (mouse, trackpad, and touchscreen) produced one cluster that was ∼97% touchscreen while the remainder were dominated by mouse-like input (Figure 1), even though touch sessions comprise only ∼11% of the corpus. The imbalance indicates that the partition was tracking acquisition physics as much as within-modality strategy. We therefore compared touchscreen and non-touchscreen sessions with Mann–Whitney U tests within each game type and observed consistent shifts across several kinematic features, which supports treating the modalities as draws from distinct generative processes rather than a single population. Consequently, the analyses that follow restrict attention to non-touchscreen sessions, leaving 26,469 sessions in the clustering cohort.

4.4 Within-Task Normalization & PCA

Task mechanics reshape the raw kinematic traces: for example, the sheepherding task exhibits the highest pause rates in our corpus, whereas thread-the-needle encourages comparatively direct paths. Because the same absolute feature values therefore carry different semantics across games, we z-score every descriptor within its game type before any cross-session comparison. Table 1 shows that several summaries are strongly correlated (*speed_mean* vs. *path_efficiency*, *duration* vs. *path_length*), which motivates a linear dimensionality reduction step. We apply principal component analysis (PCA) [1] to the within-game z-features and retain four components that jointly explain roughly 90% of the variance.

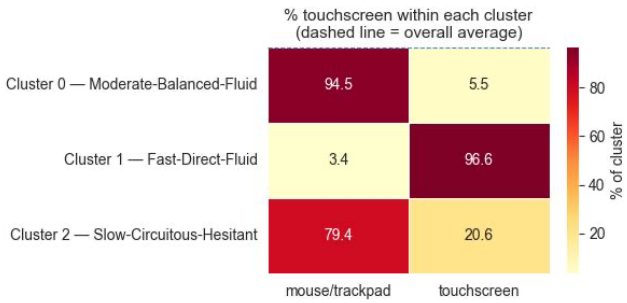

Figure 1: Percent of each cluster that is touchscreen vs. mouse/trackpad, for an initial k=3 run on *all* devices. One cluster is almost exclusively touch, motivating mouse-only segmentation.

4.5 Ensemble Clustering Pipeline

We compared k-means, Gaussian mixture models (GMM), and DBSCAN [7] on normalized features and on PCA embeddings. On PCA coordinates the diagnostics disagree: silhouette peaks near k=2, the inertia elbow lies near k=4, and Davies–Bouldin favors k=7 in our sweep. Let k_{sil}, k_{elbow}, and k_{db} denote those three choices on the PCA embedding. We set $k_{\text{med}} = \text{median}(k_{\text{sil}}, k_{\text{elbow}}, k_{\text{db}})$ and

$$K = \max(k_{\text{med}},\ 3).$$

In our run $(k_{\text{sil}}, k_{\text{elbow}}, k_{\text{db}}) = (2, 4, 7)$, hence $k_{\text{med}} = 4$ and K=4 without invoking the floor. The fitted four-cluster k-means on `X_pca` (scikit-learn `KMeans`, `random_state=42`, `n_init=10`) yields silhouette 0.246 and Davies–Bouldin 1.263 on that embedding. Reported labels use only this k-means; GMM and DBSCAN appear in the diagnostic sweep but not in the final taxonomy. Figure 2 summarizes the evaluation curves.

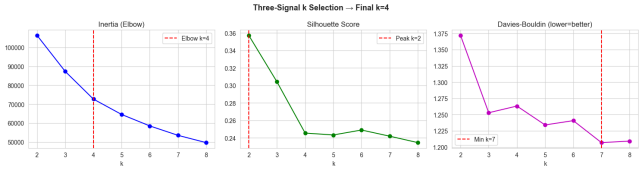

Figure 2: Ensemble cluster evaluation using Silhouette Score, Inertia, and Davies-Bouldin metrics to determine optimal $k = 4$.

4.6 Random Forest Classifier

We train a Random Forest on the six z-scored features to predict KMeans cluster membership for real-time classification. A stratified 80/20 hold-out split preserves cluster proportions.

Randomized search over 40 hyperparameter draws with five-fold stratified cross-validation in scikit-learn (objective: macro F1) selected:

- `n_estimators=500`
- `max_depth=20`
- `min_samples_leaf=1`
- `max_features=sqrt`
- `class_weight=balanced`

The tuned model reaches 97.2% accuracy on a stratified 20% hold-out (macro F1 = 0.972), with per-cluster F1 between 0.965 and 0.977; five-fold CV macro F1 is 0.969 ± 0.003. These figures measure how well the forest recovers the KMeans partition—supervised agreement with pseudo-labels—rather than accuracy against an external behavioral taxonomy. Permutation importance is largest for `path_efficiency` (0.283), followed by `speed_mean` (0.252). Gini importance agrees on these two as important drivers but weights duration more highly, indicating the feature is redundant with other kinematic summaries.

We retrain on all mouse/trackpad sessions and pickle `model.pkl`, bundling the Random Forest, PCA, KMeans centroids, per–game-type z-score stats, feature order, cluster names, and training tick rate. The Flask `/api/classify` endpoint (`POST`) downsamples live ticks, recomputes six features, z-scores, predicts, and returns PCA coordinates, per-class proabilities, and `anomaly_pct`. That score is cohort-relative distance in the PC1–PC2 plane against training points in the cohort scatter file that share the predicted cluster *and* game type. The shipped repository includes this bundle for demos and grading.

4.7 Key Contributions

Pooling sessions across games confounds task difficulty with movement strategy, so we apply z-score normalization strictly within each game type. We select K with a median ensemble over silhouette, inertia elbow, and Davies–Bouldin on the PCA embedding, which yielded K=4, and we name archetypes from relative standing among centroids using inter-cluster quartiles, so labels (e.g., *ModerateCircuitousHesitant*) reflect internal comparisons. The shipped system flags the global top 2% of sessions offline by Euclidean distance to the centroid in the PC1–PC2 plane, while live inference exposes a cohort-relative `anomaly_pct` against training points that share the predicted cluster and game type.

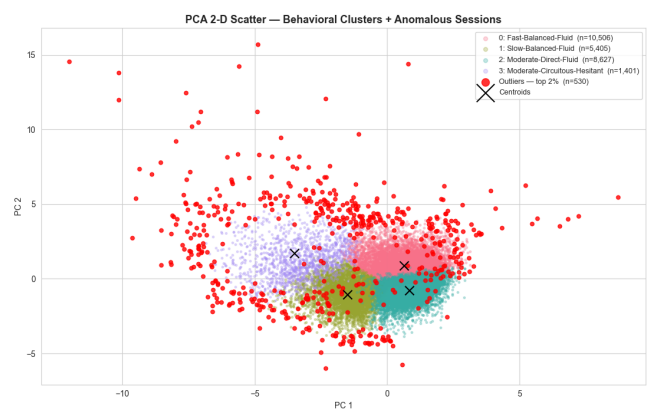

Figure 3: PCA projection highlighting the global top 2% of sessions as exploratory anomalies (centroid distance in the PC1–PC2 plane). Live sessions instead receive `anomaly_pct` relative to in-cluster, same-game training neighbors (§4).

5 Evaluation

5.1 Evaluation Questions

All experiments use CaptchaSolve30k [6], yielding 26,469 cleaned mouse and trackpad sessions. Models are implemented in Python (NumPy, pandas, scikit-learn), with live inference served through a Flask API to a D3.js frontend. Table 2 maps five questions to the observations reported below.

Table 2: Summary of Evaluation Findings

Question	Evidence / observation	Conclusion
1. Do 6 features yield interpretable profiles?	Centroid separation (Table 4); pace vs. pause.	Four interpretable archetypes.
2. Is K=4 justified?	Median vote: silhouette, elbow, DB (§4).	Selected K=4 vs. K=2 silhouette peak.
3. Does RF recover the partition?	5-fold CV macro F1 = 0.969.	RF mirrors pseudo-labels.
4. Are clusters balanced across games?	Uniform game mix (∼31–36%).	Strategy, not task-only.
5. Can the backend classify live sessions?	32 ms mean latency; ∼45 KB payload (§5.4).	Near-real-time inference.

5.2 Experimental Observations: Baseline Comparisons

Table 3 isolates preprocessing from algorithm choice on the same mouse/trackpad cohort (n=26,469, `random_state=42`). We report Silhouette and Davies–Bouldin *in the space where each model clusters* (Euclidean geometry in 6D z-features vs. 4D PCA coordinates). The first row is k-means on within-game z-scored features only (six dimensions). The second row is our reported pipeline: k-means on the four PCA components of those z-features (same K=4). The third row is GMM (`covariance_type=full`) on that same PCA embedding for comparison. Metrics match the analysis notebook and `model.pkl` export.

Table 3: Clustering baselines (Sil. = Silhouette; DB = Davies–Bouldin). Values from the final notebook run on n=26,469 sessions.

Method	Sil. (↑)	DB (↓)
k-means (within-game z, 6D, K=4)	0.225	1.361
k-means (within-game z + PCA, K=4)	0.246	1.263
GMM (PCA of z, K=4)	0.151	1.690

5.3 Experimental Observations: Cluster Profiles

Our k=4 clustering isolates distinct behavioral types. Table 4 lists each cluster's position in game-z feature space as PCA-inverse k-means centroids from the exported `model.pkl`. Values apply `inverse_transform` to the fitted cluster centers in PCA space, in the same feature order as Table 1.

The *Moderate-Circuitous-Hesitant* group (cluster 3) shows the lowest path-efficiency coordinate among centroids (−0.794), consistent with circuitous movement, together with high pause rate and duration. It remains a small fraction of the corpus (∼6%). Cross-tabulation of cluster by game type is close to uniform (∼31–36% per game within each cluster), so groups are not single-game artifacts.

Table 4: Cluster centroids in game-z space (PCA inverse of k-means centers in `model.pkl`).

Cluster	Speed	Eff.	Pause	Dur.	Len.	σ Spd
0 (Fast-Bal)	0.619	−0.585	−0.540	−0.218	0.312	0.389
1 (Slow-Bal)	−1.262	−0.125	0.935	0.258	−0.344	−0.959
2 (Mod-Dir)	0.244	0.949	−0.247	−0.422	−0.578	0.128
3 (Mod-Circ)	−1.067	−0.794	1.642	2.719	2.127	−0.003

5.4 Live Inference & Systems Metrics

We retrain the Random Forest on all sessions. To validate our claims of an operationalized system, we benchmarked the live `/api/classify` endpoint. Processing a standard 10-second trajectory (approx. 2,400 ticks, payload size ∼45 KB) requires an average end-to-end classification time of 32 ms on the Flask backend. This supports near-real-time interaction: the dashboard can refresh the PCA scatter view without noticeable delay after each classify call. The exported `model.pkl` implements the pipeline in §4.6: downsample live ticks, extract six features, z-score by game type, predict with probabilities, PCA projection, and in-cluster anomaly scoring (§4). Browser sessions use the same logic as the training export.

5.5 Interactive system

The D3.js dashboard [8, 11] is organized around three coordinated views that connect global structure, feature-level interpretation, and raw behavioral traces.

Data Exploration. The primary view is a PC1–PC2 embedding of historical training sessions, colored by cluster assignment under the same offline-trained projection. This embedding exposes the global structure of behavioral modes, allowing analysts to compare individual sessions against the broader cohort distribution.

Optional game-type filters refine the view by limiting the display to within-task subsets, enabling more focused comparisons within specific interaction contexts.

A linked feature-space view complements the embedding using radar plots of the six kinematic features. Cluster-average profiles act as behavioral archetypes, enabling direct comparison between individual sessions and their corresponding cluster signatures.

Interaction Modes. The interface supports two complementary interaction modes for exploration and detailed analysis.

Hovering over a session provides lightweight, transient comparison across views. It synchronizes highlighting between the PCA embedding and radar plots while updating session-level feature values, enabling immediate comparison against cluster-level behavior without modifying the current selection state. Figure 4 illustrates this hover-based coordination between PC space and feature space.

Selection-based interaction supports in-depth analysis of individual sessions. Selecting a point expands its associated information, triggering animated playback of the corresponding mouse trajectory and revealing its full kinematic feature profile. Figure 5 shows this selection-driven view, where raw trajectories and derived features are jointly visualized for interpretation.

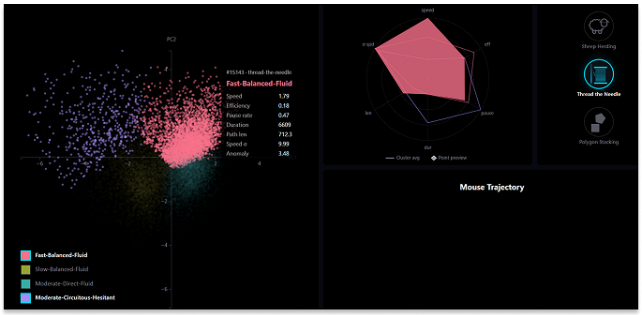

Figure 4: Coordinated PCA and radar views with game- and cluster-aware filtering. Hovering a session updates aligned feature profiles and highlights its position within the cohort structure.

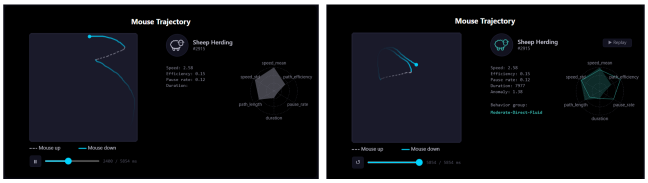

Figure 5: Selection-based interaction showing (left) mid-animation mouse trajectory playback and (right) completed trajectory with revealed cluster profile and kinematic feature values.

Live Self-Classification. Beyond retrospective exploration, the system supports real-time analysis through an embedded open-source CAPTCHA game interface adapted from the CaptchaSolve30k project. Figure 6 shows the interface, consisting of a landing page with available game modes (left) and an in-session gameplay screenshot (right).

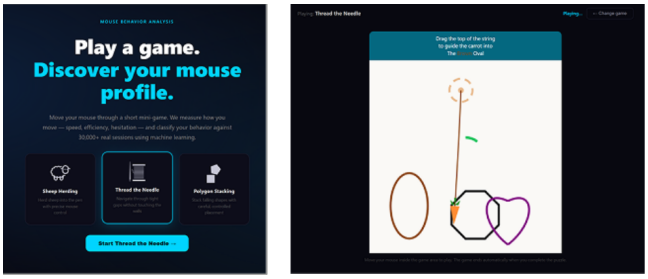

Figure 6: Live self-classification interface with embedded CAPTCHA game for real-time behavioral inference. Left: landing page with game options. Right: in-session gameplay screenshot showing active user interaction. The game component is adapted from the CaptchaSolve30k project.

During gameplay, user interactions are streamed as raw mouse trajectories to a Flask backend. These trajectories are processed online, where a trained random forest classifier assigns each session to a behavioral cluster. The prediction is immediately reflected in the PCA embedding and feature-space summaries, and enables on-the-fly trajectory playback. This forms a real-time feedback loop in which live CAPTCHA interactions are continuously mapped into the learned behavioral space and compared against historical data.

6 Conclusions and Discussion

In this work, we showed that median-consensus k-means on PCA-reduced, task-normalized kinematic features yields compact behavioral archetypes with separated centroids and Silhouette score 0.246 on the retained embedding. Cross-tabulations of cluster membership against game type remain close to uniform, which indicates that the recovered modes reflect movement strategy rather than single-game artifacts. We packaged the resulting models in a lightweight Flask service (32 ms end-to-end latency on representative payloads) and linked them to a D3.js dashboard so offline and live sessions share a common projection for qualitative inspection.

Several limitations remain. The study targets *human behavioral segmentation* rather than bot detection, and without adversarial ground truth the anomaly statistic identifies statistical extremes in the PC plane rather than validated fraud. The Random Forest is a surrogate for the k-means partition (macro F1 = 0.967) and does not by itself generalize beyond the four archetypes estimated here; touchscreen data are also excluded from the current cohort. Future work includes cross-seed stability analyses, richer device coverage, and supervised baselines once adversarial trajectory corpora become available.

Team Effort

All members have contributed comparably across data engineering, modeling, visualization, and writing.

References

[1] Hervé Abdi and Lynne J. Williams. 2010. Principal Component Analysis. *WIREs Computational Statistics* 2, 4 (2010), 433–459.

[2] Saeed Aghabozorgi, Ali Seyed Shirkhorshidi, and Teh Ying Wah. 2015. Time-Series Clustering—A Decade Review. *Information Systems* 53 (2015), 16–38.

[3] Ahmed Awad E. Ahmed and Issa Traore. 2007. A New Biometric Technology Based on Mouse Dynamics. *IEEE Transactions on Dependable and Secure Computing* 4, 3 (2007), 210–223.

[4] Donald J. Berndt and James Clifford. 1994. Using Dynamic Time Warping to Find Patterns in Time Series. In *Proceedings of the AAAI Workshop on Knowledge Discovery in Databases*. AAAI Press, Menlo Park, CA, 359–370.

[5] Elie Bursztein, Matthieu Martin, and John C. Mitchell. 2011. Text-Based CAPTCHA Strengths and Weaknesses. In *Proceedings of the 18th ACM Conference on Computer and Communications Security (CCS '11)*. ACM, New York, NY, 517–528.

[6] CAPYCAP Inc. 2025. CaptchaSolve30k: The Largest Open-Source CAPTCHA Interaction Dataset. Hugging Face Datasets.

[7] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. 1996. A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining*. AAAI Press, Menlo Park, CA, 226–231.

[8] Jeffrey Heer and Ben Shneiderman. 2012. Interactive Dynamics for Visual Analysis. *Queue* 10, 5 (2012), 30–31.

[9] Daniel Keim, Gennady Andrienko, Jean-Daniel Fekete, Carsten Görg, Jörn Kohlhammer, and Guy Melançon. 2008. Visual Analytics: Definition, Process, and Challenges. In *Information Visualization: Human-Centered Issues and Perspectives*, Andreas Kerren, John T. Stasko, Jean-Daniel Fekete, and Chris North (Eds.). Lecture Notes in Computer Science, Vol. 4950. Springer, Berlin, Heidelberg, 154–175.

[10] Leland McInnes, John Healy, and James Melville. 2020. UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. arXiv:1802.03426

[11] Tamara Munzner. 2014. *Visualization Analysis and Design*. CRC Press, Boca Raton, FL.

[12] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. 2017. Vega-Lite: A Grammar of Interactive Graphics. *IEEE TVCG* 23, 1 (2017), 224–236.

[13] Chao Shen, Zhongmin Cai, Xiaohong Guan, Yajun Du, and Roy A. Maxion. 2013. User Authentication Through Mouse Dynamics. *IEEE Transactions on Information Forensics and Security* 8, 1 (2013), 1–15.

[14] Giuseppe Stragapede, Ruben Vera-Rodriguez, Ruben Tolosana, and Aythami Morales. 2023. BehavePassDB: Public Database for Mobile Behavioral Biometrics and Benchmark Evaluation. *Pattern Recognition* 140 (2023), 109516.

[15] Luis von Ahn, Manuel Blum, Nicholas J. Hopper, and John Langford. 2003. CAPTCHA: Using Hard AI Problems for Security. In *Advances in Cryptology – EUROCRYPT 2003 (Lecture Notes in Computer Science, Vol. 2656)*. Springer, Berlin, Heidelberg, 294–311.

[16] Luis von Ahn, Benjamin Maurer, Colin McMillen, David Abraham, and Manuel Blum. 2008. reCAPTCHA: Human-Based Character Recognition via Web Security Measures. *Science* 321, 5895 (2008), 1465–1466.

[17] Nan Zheng, Aaron Paloski, and Haining Wang. 2011. An Efficient User Verification System via Mouse Movements. In *Proceedings of the 18th ACM Conference on Computer and Communications Security (CCS '11)*. ACM, New York, NY, 201–212.